مدينة زويل للعلوم والتكنولوجيا
Zewail City of Science and Technology

Interactive Reinforcement Learning Algorithm Visualizer

1. Introduction

The application acts as a learning platform to learn model-based and model-free RL algorithms via the simultaneous visualization of their learning processes. The tool deploys six essential RL algorithms in two different environments, allowing tuning of the parameters and employing different ways of visualizing the results to make learning and experimenting easier and more fun.

2. Implemented Environments

GridWorld Environment: The GridWorld environment consists of a 3×3 grid with the following characteristics:

- State Space: 9 discrete states (S1-S9) arranged in a grid
- Goal State: S9 (bottom-right) with a reward of +100
- Start State: S1 (top-left)
- Transition Dynamics: 80% success probability for intended moves, 20% probability of staying in place
- Reward Structure: -1 per movement step, +100 for reaching the goal
- Terminal States: S9 (goal)
- Actions: Four directional movements (up, down, left, right)

Frozen Lake 4×4 Environment:

- State Space: 16 discrete states arranged in a 4×4 grid
- Goal State: S15 (bottom-right) with reward +1
- Start State: S0 (top-left)
- Hole States: S5, S7, S11, S12 with reward -1 (terminal states)
- Safe States: Remaining states are frozen ice with reward 0
- Transition Dynamics: Deterministic movement (100% success probability)
- Actions: Four directional movements (up, down, left, right)

3. Implemented Reinforcement Learning Algorithms

3.1Model-Based Algorithms

3.1.1Value Iteration

Value iteration is a dynamic programming algorithm that iteratively updates state value estimates until convergence.

- State values are set to zero as a starting point.
- A user interface allows for adjustment of the convergence threshold (θ).
- For safety of computation, the maximum number of iterations is limited to 1000.
- Intermediate value and policy estimates are stored for the purpose of visualization.

3.1.2 Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement:

1. Policy evaluation computes the value function for the current policy
2. Policy improvement updates the policy to be greedy with respect to the computed values

- Start with random policy
- Separate thresholds for convergence of the evaluation and improvement process
- Clear distinguishing patterns between the evaluate and improve phase

3.2 Model-Free Algorithms

3.2.1 SARSA

SARSA is an on-policy temporal difference learning algorithm that updates Q-values using the actual next action taken.

- ε-greedy exploration strategy with exponential decay
- Small random values were used for the initialization of the Q-table
- Learning through episodes with the number of episodes that can be set as per requirement
- Tracking of success and gathering of reward statistics were done

3.2.2 Q-Learning

Q-learning is an off-policy temporal difference algorithm that uses the maximum Q-value of the next state

3.2.3 Monte Carlo Control (First Visit)

First-visit Monte Carlo control approximates Q-values with full episode returns:

- The first appearance of every state-action pair in one episode is the only one that counts in the average return
- The returns are computed from the initial visit to the end of the episode
- The sample means are responsible for updating the Q-values after every episode

3.2.4 Monte Carlo Control (Every Visit)

Each visit in Monte Carlo control considers every occurrence of state-action pairs:

- All visits state-action pairs are included in the calculation of return averages
- Might have more data for each episode but also with greater variance

4. Parameter Adjustment Capabilities

Common Parameters:

- Gamma (γ): Discount factor
- Theta (θ): Convergence threshold

Model-Free Specific Parameters

- Episodes: Number of learning episodes.
- Epsilon (ε): Exploration rate for ε-greedy policies.
- Alpha (α): Learning rate for temporal difference algorithms.

Adaptive Features

- Automatic parameter adjustment according to the chosen algorithm
- Dynamic interface displaying/hiding corresponding parameters
- Suggested value ranges indicated via tooltips
- Parameter verification with suitable limits

5. Visualization Techniques

Grid Visualization: The grid employs a combination of colors, arrows, and highlights to effectively indicate state types, policy decisions, and the agent's movement instantaneously.

Graph Visualization (D3.js): The graph is capable of dynamically displaying nodes, transitions, and changing size to accurately depict state types, values, and terminal states.

Textual Displays: Outputs in JSON format, lists of policies, and logs constitute a structured and easy-to-read way of showing values, actions, and events of the algorithm.

Progress and Statistics: The learning progress and performance during training are monitored by iteration steps, delta changes, counters, and agent statistics.

Interactive Features: Users are provided with the means to take control of the algorithm's steps, run animations, switch between modes, and adjust parameters to explore and compare algorithms interactively and more easily.